

Pythonic Argo: Building Maintainable and Dynamic ML Pipelines as Code

Alibaba Cloud, Shuangkun Tian & Lei Hou



Content

目录

01 ML Meets Argo Workflows

02 Moving from YAML to Python

03 Building Type-Safe & Dynamic Workflows in Python

04 Best Practices for Production-Ready ML Pipelines

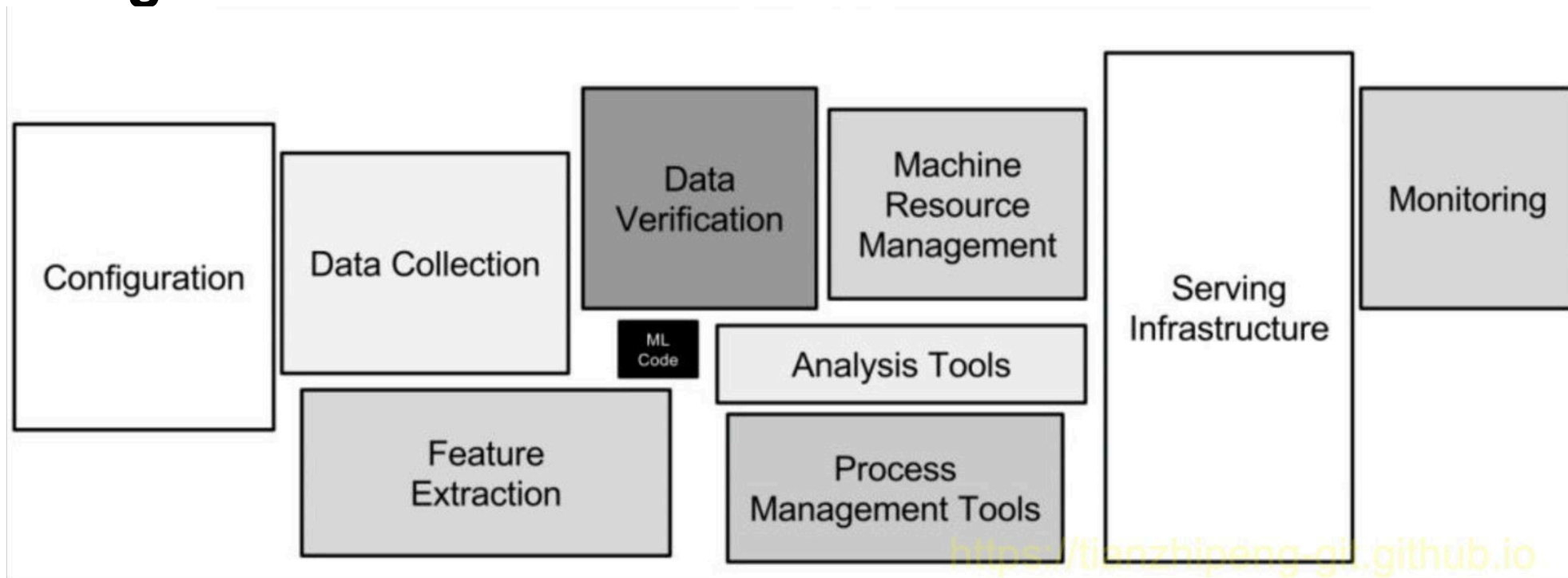


Part 01

ML Meets Argo Workflows



ML Engineers' Core Pain Points

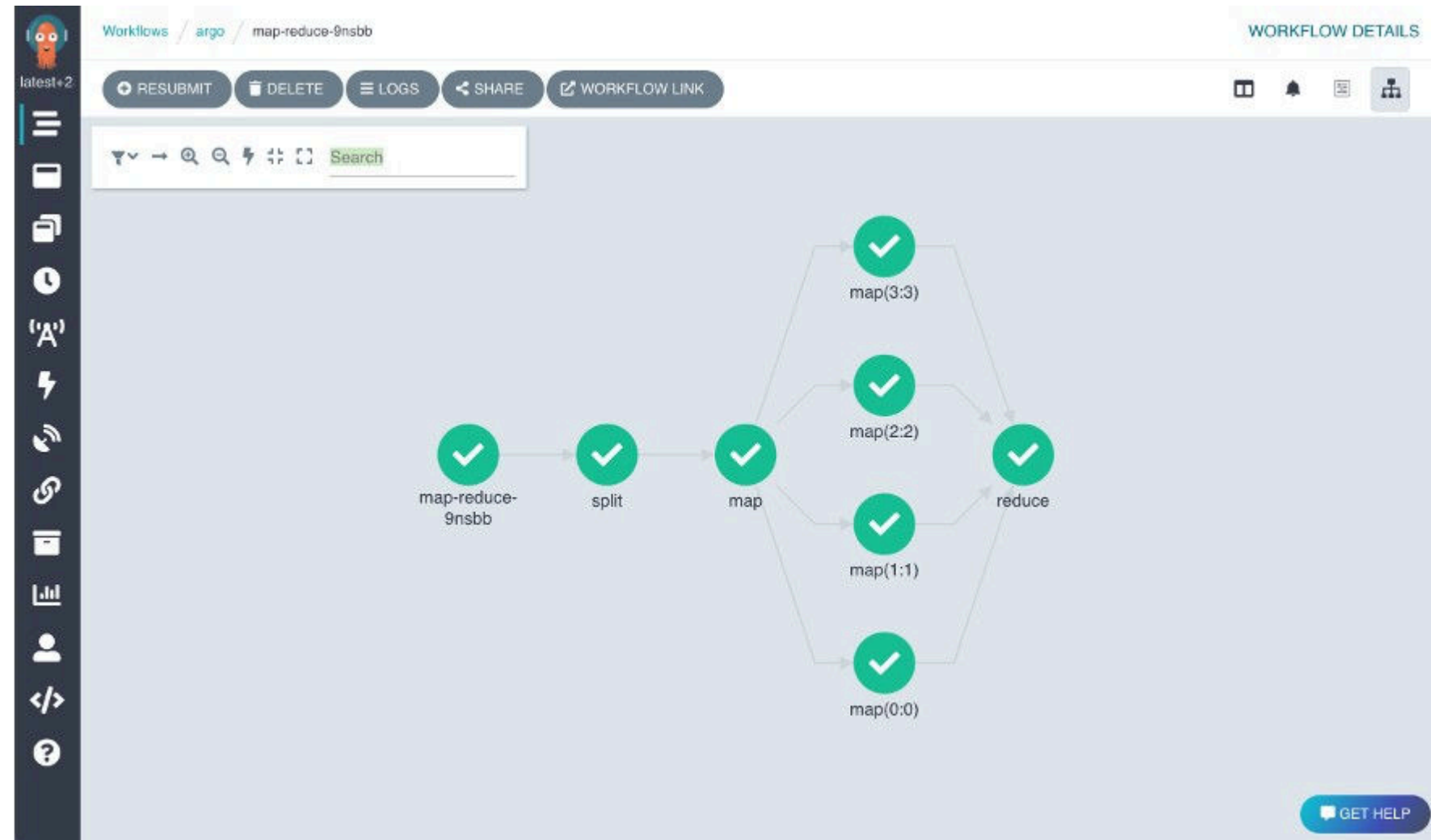


Orchestration: Connecting the pipeline

- With petabytes of prepared training data and models needing retraining on schedules, coordinating multi-step workflows becomes critical. A typical ML pipeline involves Spark preprocessing, distributed training across thousands of GPUs, model validation, and model deployment. Running these manually doesn't scale.
- **Kubeflow Pipelines** provides portable ML workflows with experiment tracking. **Argo Workflows** enables complex DAGs spanning Spark jobs, PyTorch training, and KServe deployments. The orchestration layer transforms ad-hoc scripts into production pipelines that trigger retraining when data drift is detected

ML Engineer and Argo Workflows

- Underlying supporter of many AI/ML orchestration frameworks
 - Kubeflow Pipeline
 - Metaflow
 - Kedro
- Also one of the most popular MLOps tools
 - CNCF Graduation
 - Ranked highly in the MLOps Radar Quadrant, 90% recommendation rate

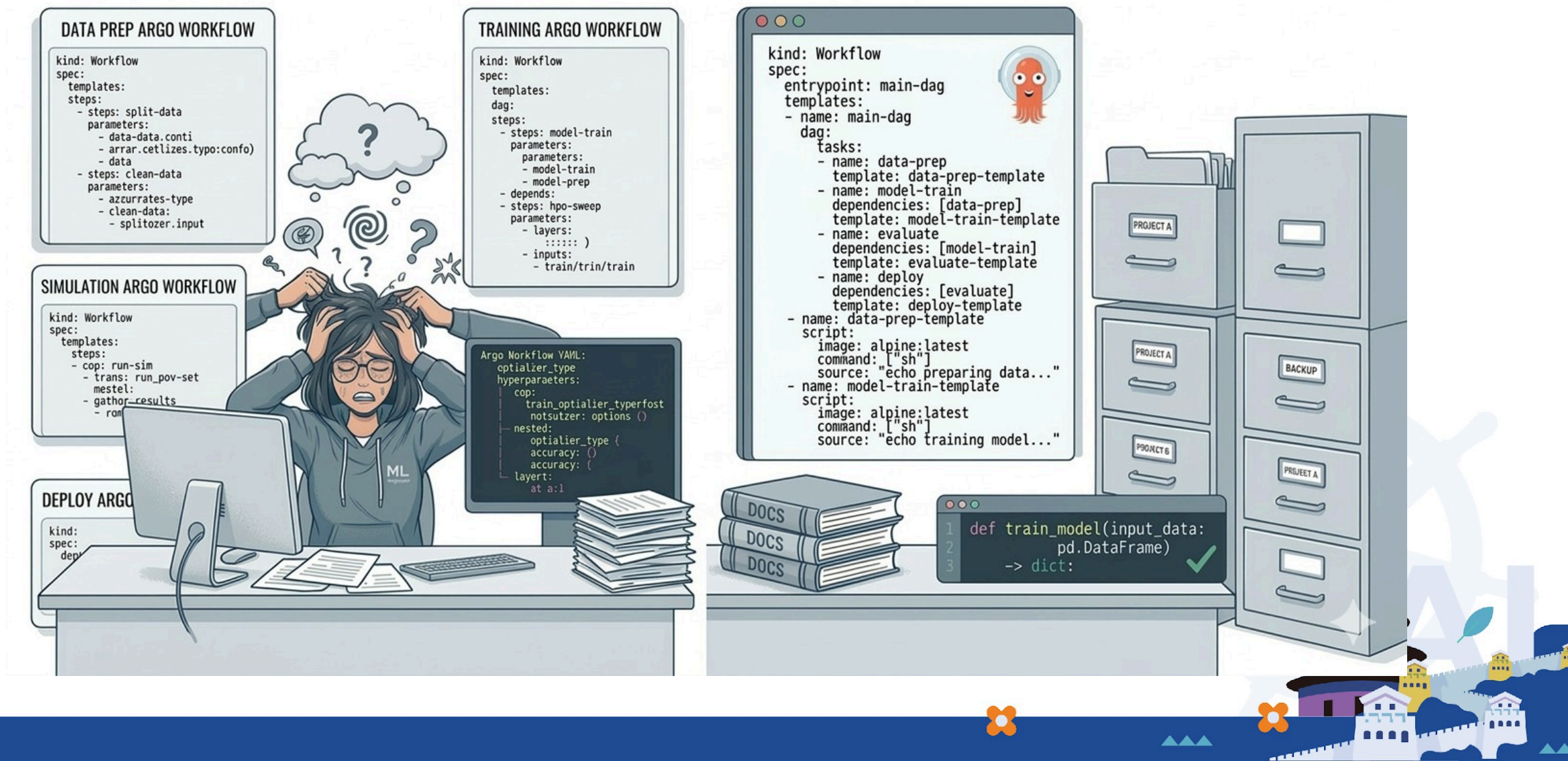


Part 02

Moving from Static YAML to Dynamic Python



The Friction: When ML Engineer Meets YAML

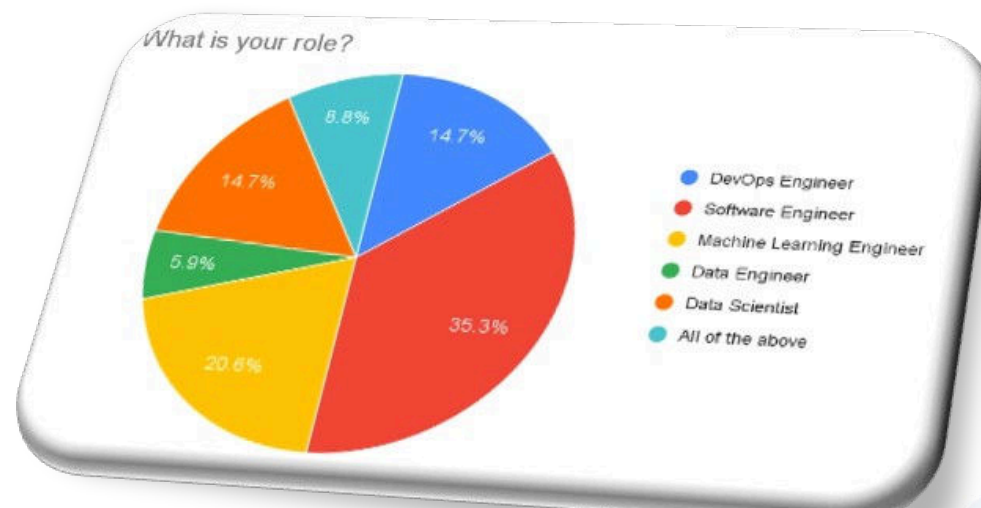


Hera Bridging the Gap: Workflow-as-Code

Python Orchestration

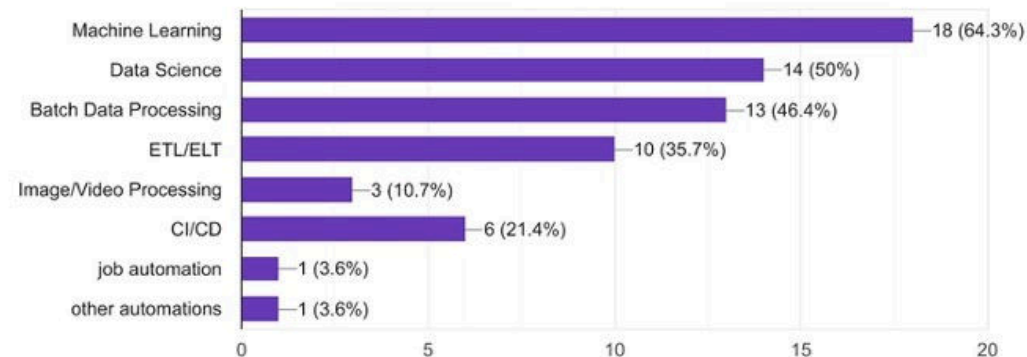
Powered by Argo Workflows

- ✓ **Python-First:** Write native Python (with all your favorite libraries!)
- ✓ **Lightweight & Unintrusive:** Keep orchestration logic out of your functions
- ✓ **Built on Argo Workflows:** All the power of Kubernetes, with the simplicity of Python



What are your main use cases when using Hera?

28 responses



Seamless Conversion: Python Functions as Task

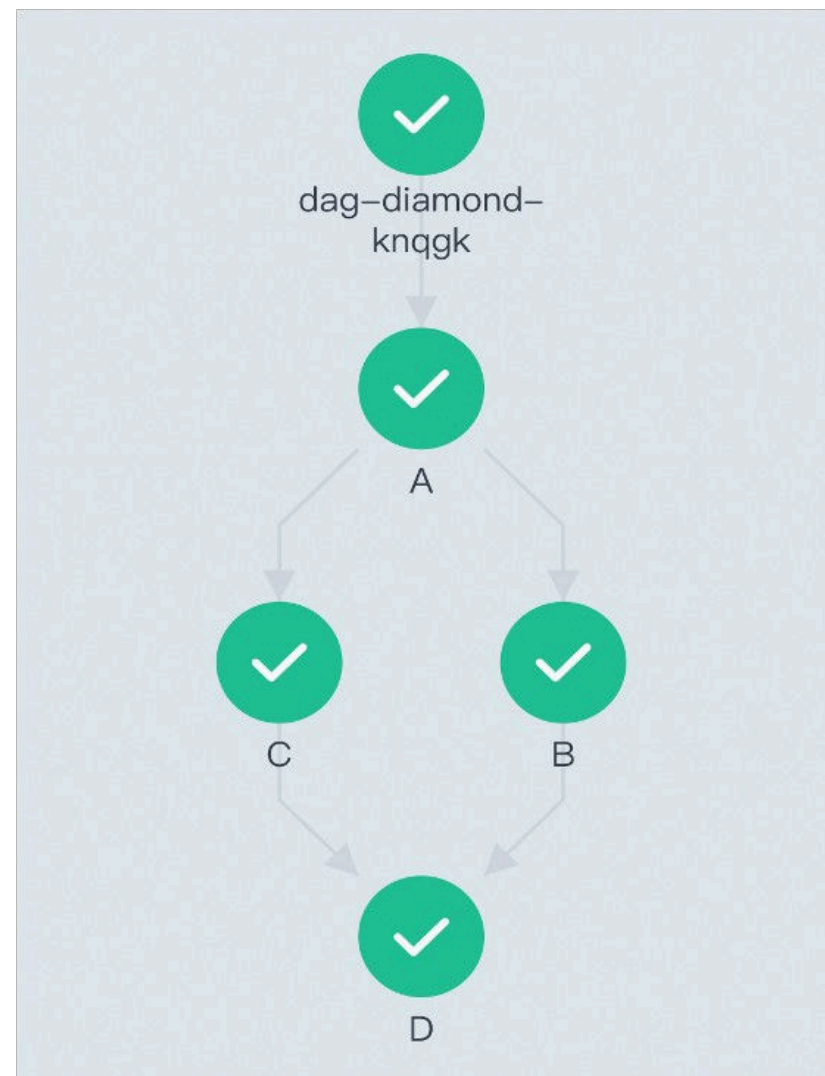
```
from hera.workflows import DAG, Workflow, script

@script()
def echo(message: str):
    print(message)

with Workflow(
    generate_name="dag-diamond-",
    entrypoint="diamond",
) as w:
    with DAG(name="diamond"):
        A = echo(name="A", arguments={"message": "A"})
        B = echo(name="B", arguments={"message": "B"})
        C = echo(name="C", arguments={"message": "C"})
        D = echo(name="D", arguments={"message": "D"})
        A >> [B, C] >> D # Define execution order

w.create()
```

python dag.py



Part 03

Building Type-Safe & Dynamic Workflows in Python



Type-Safe Data Passing with Python Hints

```
def backtest(
    policy_code: str,
    start_date: str = "20210316",
    end_date: str = "20260313"
):
    """
    Args:
        policy_code: Policy code
        start_date: Start date, format YYYYMMDD
        end_date: End date, format YYYYMMDD
    """
```

Origin Code

```
def backtest_template():
    return Script(
        name="backtest",
        image="python:3.11-slim",
        inputs=[
            Parameter(name="stock_code"),
            Parameter(name="start_date", value="20210316"),
            Parameter(name="end_date", value="20260313"),
        ],
        outputs=[
            OSSArtifact(
                name="raw-data",
                path="/tmp/raw-data.json",
                archive=NoneArchiveStrategy(),
                key="{{workflow.name}}/raw/{{inputs.parameters.stock_code}}.json",
            ),
        ],
        source=backtest_stock_data,
    )
```

Define Script

- Input
- OutPut
- Environment
- Python Function

```
templates:
- name: backtest
  inputs:
    parameters:
      - name: policy_code
      - name: start_date
        value: '20210316'
      - name: end_date
        value: '20260313'
  outputs:
    artifacts:
      - name: raw-data
        path: /tmp/raw-data.json
        archive:
          none: {}
        oss:
          key: '{{workflow.name}}/raw/{{inputs.parameters.policy_code}}.json'
  script:
```

Template in Workflow



Dynamic & Parallel

Parallel Code In Python

```
with DAG(name="main"):
    for policy_code in policy_codes:
        backtest_task = Task(
            name=f"backtest-{policy_code}",
            template=back_test_template,
            arguments={
                "policy_code": policy_code,
                "start_date": "20210316",
                "end_date": "20260313",
            },
        )
        calc_task = Task(
            name=f"calc-{policy_code}",
            template=calc_template,
            dependencies=[f"backtest-{policy_code}"],
            arguments={
                "policy_code": policy_code,
                "raw-data": backtest_task.get_artifact("raw-data"),
            },
        )
```

Parallel Code In DAG

```
dag:
  tasks:
    - name: backtest-000001 ...
    - name: calc-000001 ...
    - name: backtest-000300 ...
    - name: calc-000300 ...
    - name: backtest-000905 ...
    - name: calc-000905 ...
```



Live Demo



Backtesting

Backtest effect



Effect evaluation

```
{
  "sharpe_ratio": 0.38936926405596306,
  "max_drawdown": -0.4180800784441893,
  "longest_recovery_days": 1154,
  "data_points": 1210
}
```

Analysis



Performance Analysis Report

Report Date: March 20, 2026

Data Source: Historical trading data statistics

Analysis Targets: 000001, 000300, 000905

1. Overall Market Overview

Based on 1,210 data points (covering approximately 5 years of trading days), the market during this analysis period exhibits characteristics of **high volatility and prolonged recovery**.

- **Significant divergence in risk-return ratios:** Among the three targets, only **000001** and **000905** achieved positive Sharpe Ratios (> 0), indicating they

Live Demo



report_generator.py X workflow.yaml

generator > report_generator.py

```
237
238
239 def run_report_generation(policy_codes: List[str]):
240     """
241     主函数：执行完整的报表生成流程
242     """
243     print(f"开始处理 {len(policy_codes)} 个策略: {policy_codes}")
244
245     # 1. 创建工作流
246     workflow = create_stock_workflow(policy_codes)
247     workflow.to_yaml()
248     # 2. 提交工作流到 Argo
249     print("提交工作流到 Argo...")
250     workflow.create()
251
252     # 3. 等待工作流完成
253     print("等待工作流执行完成...")
254     workflow.wait()
255
256     # 4. 从 OSS 拉取结果
257     print("从 OSS 拉取分析结果...")
258     results = {}
259     import oss2
260
261     auth = oss2.Auth(OSS_ACCESS_KEY, OSS_SECRET_KEY)
```

问题 输出 调试控制台 终端 端口 2

o (test) hl@iZj6c7hj775d4rnlnqljilZ:~/workspace/test/generator\$

+ ... | ☐ X

bash gener...

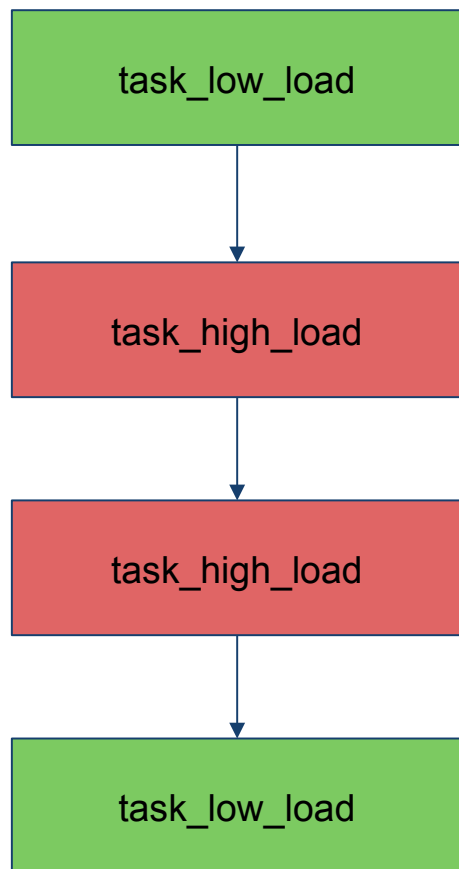
python3 g...

Part 04

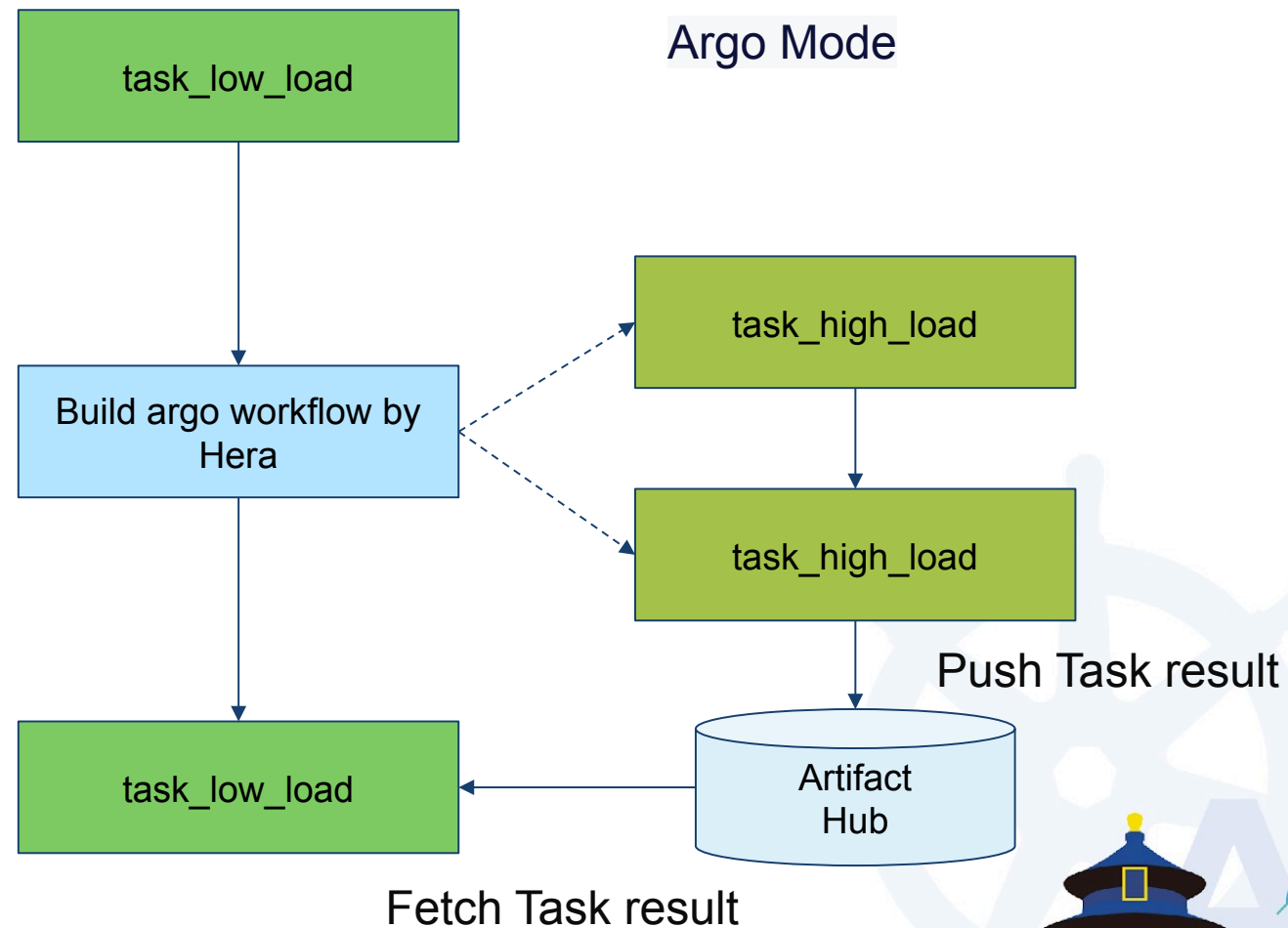
Best Practices for Production-Ready ML Pipelines



Production-Ready Checklist



Traditional Mode



Argo Mode



Thanks.

